Guia de integração — segundo display do motor

Contrato das três telas do firmware atual, conferido em 13/11/2026. A integração usa as APIs existentes da LinkerWX.

Referência visual: imagens das telas e legenda de cada controle.

1. Identidade e conexão LinkerWX

Parâmetro	Valor
Grupo Linker	**1**
ID sugerido para o segundo display	**3**
Display existente	Grupo **1**, ID **2**
Placa do motor	Grupo **1**, ID **1**
Chave de criptografia	**3 bytes:** `0xFF, 0xFE, 0x84`
Token de registro do exemplo atual	`987654321`
Servidor / porta	`LinkerWX::SERVER_ADDRESS` / `LinkerWX::SERVER_PORT`
TLS	Usar o mesmo `ROOT_CA` de `fluxo.h`; configurar o Wi-Fi da instalação.

Trecho de configuração da identidade e da chave:

```
clientLinker.setup(3, 1, "987654321"); // ID, grupo, token
const uint8_t cryptKey[] = {0xFF, 0xFE, 0x84};
clientLinker.setCryptKey(cryptKey, sizeof(cryptKey));
```

IDs devem ser exclusivos no grupo; reservar o ID 3 ao segundo display. A chave contém bytes, não o texto ASCII `"FFFE84"`.

2. Comandos enviados por todas as telas

Cada comando gera **um broadcast ao grupo 1**, com `SendType::ONLY_SEND`. Os exemplos são o payload completo; a LinkerWX monta o cabeçalho.

Comando	Origem na interface	Significado no motor	Payload completo de exemplo
`START`	START principal; partida/repartida dos programas	Partida direta; Modbus `0x2000 = 1`.	`cmd=START`

Comando	Origem na interface	Significado no motor	Payload completo de exemplo
`STOP`	STOP de qualquer tela; fim do tempo ligado dos programas	Parada suave; Modbus `0x2000 = 6`.	`cmd=STOP`
`REVERSE`	REVERSE da tela principal	Funcionamento no sentido reverso; Modbus `0x2000 = 2`.	`cmd=REVERSE`
`EMERGENCY`	EMERGENCY da tela principal	Solicita emergência; Modbus `0x2000 = 5`.	`cmd=EMERGENCY`
Inteiro `N`, de 1 a 100	Slider geral; partida de cada ciclo de Prog1/Prog2	Referência de **N Hz**; Modbus `0x1000 = N × 100`.	`cmd=50`

Exemplo de chamada completa:

```
clientLinker.broadcastText(1, "cmd=50", SendType::ONLY_SEND);
```

Regras do display atual:

- Respeitar **500 ms entre broadcasts**, sem duplicar por destinatário. A fila atual comporta 15 comandos; STOP/EMERGENCY não têm prioridade especial.
- Slider geral: consome a última edição positiva pendente a cada 1 s, conectado e sem programa local ativo; não reenvia sem nova edição.
- **Slider N significa N Hz**, apesar do `%` visual. Zero não é enviado e não equivale a STOP.
- Offline/reconectando não enfileira comandos; mudança de sessão descarta os pendentes. `ONLY_SEND` não confirma execução pelo motor.
- PROG 1 / PROG 2 da tela principal e setas apenas navegam. Campos e sliders de programa alteram parâmetros locais.

3. Prog1 e Prog2

Mesma lógica, parâmetros separados e **uma rotina ativa por display**. Os temporizadores são locais; não existe `cmd=PROG1` ou `cmd=PROG2`.

Parâmetro	Prog1: valor inicial	Prog2: valor inicial
Espera antes da primeira partida	10 s	10 s
Tempo ligado	20 s	30 s
Tempo parado entre ciclos	10 s	30 s
Frequência máxima do formulário	60 Hz	60 Hz
RPM máximo do formulário	1798 RPM	1798 RPM

Parâmetro	Prog1: valor inicial	Prog2: valor inicial
Slider de frequência	0; escolher valor positivo	0; escolher valor positivo

Sequência ao pressionar o botão de execução na tela do programa:

1. Exigir conexão, copiar os tempos e aguardar a espera inicial.
2. Enfileirar `cmd=N` **e depois** `cmd=START`. O par é aceito inteiro ou recusado; transmissões separadas por pelo menos 500 ms.
3. Contar o tempo ligado e enfileirar `cmd=STOP`.
4. Contar o tempo parado e repetir o passo 2, até cancelamento ou perda da sessão.

Exemplo de Prog1 com slider em 50, sem fila anterior e com rede disponível:

```
t=0 s     Iniciar programa; aguardar 10 s.
t≈10 s    Enfileirar cmd=50 e cmd=START; transmitir cmd=50.
t≥10,5 s  Transmitir cmd=START, respeitando os 500 ms.
t≈30 s    Enfileirar/transmitir cmd=STOP.
t≈40 s    Repetir cmd=50, depois cmd=START; continuar os ciclos.
```

Tempos contam desde a inserção na fila, sem confirmação física. Fila/rede afetam a duração real no motor. No Prog2 padrão, a primeira parada é solicitada em aproximadamente 40 s e a próxima partida em 70 s, nas mesmas condições.

Comportamentos que a segunda interface deve conhecer:

- STOP da tela de programa cancela a rotina e solicita **um STOP**. STOP/EMERGENCY principais **não cancelam** a rotina local.
- Navegar mantém a rotina. Iniciar outro programa substitui a execução e reinicia a espera, sem STOP automático.
- Tempos são copiados ao iniciar o programa. Slider editado vale na próxima partida do ciclo.
- Queda de sessão, setpoint zero ou falha de inserção cancelam a rotina local. Reconexão exige nova partida; cancelamento local não comprova parada física.
- Frequência máxima não limita/converte `N`. `SET RPM = inteiro(slider / 100 × RPM máximo)` é visual: slider 50 e RPM máximo 1798 mostram 899, mas enviam `cmd=50`.
- Programas e parâmetros não são sincronizados entre displays. Evitar rotinas simultâneas nos dois, pois ambos comandam o mesmo motor.

4. Telemetria recebida do motor

Origem: **grupo 1, ID 1**. Broadcast ao grupo 1 com `SendType::REQ_ACK`, após leitura Modbus bem-sucedida, normalmente a cada aproximadamente 1 s.

```
fa=45.0;fs=50.0;vdc=310.0;vs=220.0;i=2.3;tq=65.0;edo=1;rpm=1350
```

Campo	Significado / uso
`fa`	Frequência atual, Hz; texto.
`fs`	Referência de frequência lida do inversor, Hz; texto.

Campo	Significado / uso
`vdc`	Tensão DC, V; barra.
`vs`	Tensão de saída, V; barra.
`i`	Corrente de saída, A; barra.
`tq`	Torque estimado, %; armazenado.
`edo`	Máscara de bits das saídas digitais; armazenada.
`rpm`	Rotação do sensor, RPM; gauge e série 0 dos três gráficos.

Separadores: `;` entre campos e `=` entre chave/valor; decimais com ponto. Valores já estão em unidades físicas. As barras atuais usam inteiros. Sem amostra válida por 5 s ou após queda, frequências mostram `--`; medidores mantêm a última leitura e gráficos deixam de receber pontos antigos.

5. Receber `cmd=N` para sincronizar o slider geral

No segundo display (**ID 3**), aceitar **grupo 1, ID 2**. O display existente aceita o ID 3. Ignorar a própria origem e outros dispositivos nessa função.

Exemplo recebido: `cmd=50`.

1. Em `onRECV`, validar `cmd=` seguido de inteiro completo entre **0 e 100**. Outros comandos, negativos, decimais ou texto misturado são ignorados.
2. Gravar **flag + valor**, protegidos entre tarefas e vinculados à sessão conectada. Conservar o valor mais recente; descartar atualização de sessão anterior.
3. Na tarefa gráfica, consumir a flag e aplicar:

```
slidergeral.setValue(N, false);       // Sem callback.
hslider0_val = slidergeral.getValue();
labelhslidergе.setTextInt(hslider0_val);
```

Não chamar o callback manual nem criar envio/eco. Pendências de edições locais anteriores não são canceladas automaticamente.

`cmd=N` sincroniza **somente o slider geral e sua label**, inclusive quando enviado por um programa. Não muda parâmetros de Prog1/Prog2 nem inicia rotinas. É um valor solicitado; `fs` é a referência lida do inversor e atualmente atualiza apenas seu texto, sem reposicionar o slider. Não há envio `freq=N`.

6. Continuidade da interface

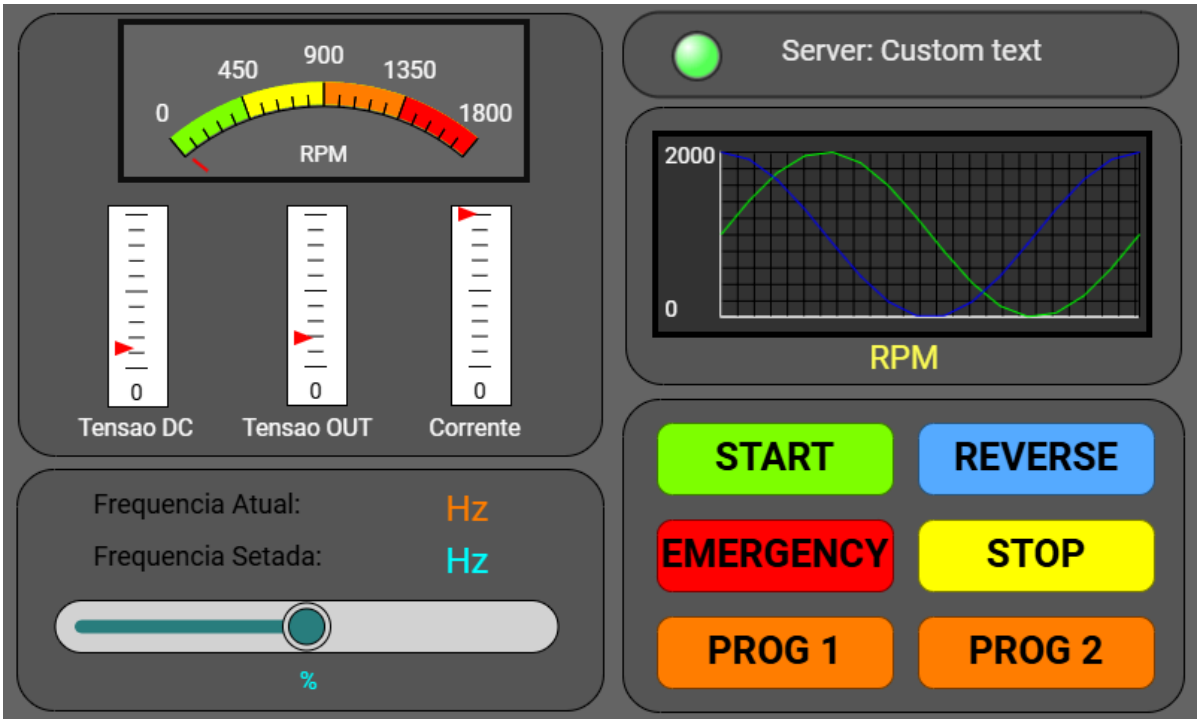
Interface ativa sem Wi-Fi/Linker; rede em tarefa separada. Após falha/queda, esperar 60 s para nova tentativa; registro tem prazo de 10 s após abrir o transporte. Watchdog das tarefas: timeout comum de 240 s. Widgets pertencem à tarefa gráfica.

Status local: **Conectando/amarelo**, **Conectado/verde**, **Offline/vermelho**. Conectado representa registro no Linker, sem comprovar execução no motor.

7. Imagens e legendas das telas

As capturas abaixo são referências do editor de interface. Textos como `Server: Custom text`, valores e curvas são ilustrativos. As legendas descrevem o firmware atual: o status mostra Conectando/Conectado/Offline e os gráficos recebem RPM somente na série 0. Marcadores de seleção do editor, visíveis ao redor de STOP na captura de Prog1, não são controles da aplicação.

7.1 Tela principal — Screen0



Campo ou botão / localização	**Tipo**	**O que representa ou faz**
Medidor RPM, superior esquerdo	Leitura	Rotação recebida em `rpm`; escala de 0 a 1800 RPM.
Tensão DC, primeira barra à esquerda	Leitura	Campo `vdc`, em volts; escala de 0 a 400 V.
Tensão OUT, barra central	Leitura	Campo `vs`, em volts; escala de 0 a 270 V.
Corrente, terceira barra	Leitura	Campo `i`, em amperes; escala de 0 a 10 A.
Frequência Atual, inferior esquerdo	Leitura	Campo `fa`, em Hz; mostra `--` quando a telemetria perde validade.
Frequência Setada, abaixo da frequência atual	Leitura	Campo `fs`, referência lida do inversor, em Hz; também expira para `--`.

Campo ou botão / localização	Tipo	O que representa ou faz
Slider geral e valor `%` abaixo, inferior esquerdo	Ajuste e leitura do ajuste	Seleciona `N`; envia `cmd=N` conforme as regras da seção 2. Também recebe a atualização visual do outro display. O `%` é apresentação: `N` é enviado em Hz.
LED e texto Server, superior direito	Status local	`led1` e `labelservstat`: amarelo/Conectando, verde/Conectado, vermelho/Offline.
Gráfico RPM, à direita, acima dos botões	Leitura	Histórico de `rpm`; escala vertical de 0 a 2000 RPM.
START, botão verde	Comando	Solicita partida direta: `cmd=START`.
REVERSE, botão azul	Comando	Solicita sentido reverso: `cmd=REVERSE`.
EMERGENCY, botão vermelho	Comando	Envia `cmd=EMERGENCY`; não cancela o programa local ativo.
STOP, botão amarelo	Comando	Envia `cmd=STOP`; não cancela o programa local ativo.
PROG 1, botão laranja inferior esquerdo do grupo	Navegação	Abre a tela de Prog1; não inicia a rotina nem envia comando.
PROG 2, botão laranja inferior direito	Navegação	Abre a tela de Prog2; não inicia a rotina nem envia comando.

7.2 Tela de Prog1 — Screen1



Nesta tela, o botão laranja **PROG 1 executa o programa**, diferentemente do botão de navegação da tela principal. Tempos iniciais: espera de 10 s, ligado por 20 s e parado por 10 s.

7.3 Tela de Prog2 — Screen2



Nesta tela, o botão laranja **PROG 2 executa o programa**. Tempos iniciais: espera de 10 s, ligado por 30 s e parado por 30 s. Os parâmetros são independentes dos de Prog1.

A legenda abaixo vale para **ambas as telas de programa**, que têm os mesmos campos e funções:

Campo ou botão / localização	**Tipo**	**O que representa ou faz**
Seta azul, canto superior esquerdo	Navegação	Retorna à tela principal sem cancelar a rotina ativa.
Tempo de início, primeiro campo à esquerda	Editável, segundos	Espera antes da primeira partida. É copiado ao iniciar o programa.
Tempo ligado, segundo campo	Editável, segundos	Tempo local entre enfileirar a partida e solicitar STOP.
Tempo parado, terceiro campo	Editável, segundos	Espera após solicitar STOP, antes de começar o próximo ciclo.
Freq. máxima, quarto campo	Editável, Hz	Parâmetro inicialmente em 60; o firmware atual não o usa para limitar ou converter o comando `N`.
RPM máxima, quinto campo	Editável, RPM	Referência do cálculo visual de SET RPM; valor inicial 1798.

Campo ou botão / localização	Tipo	O que representa ou faz
SET RPM, texto abaixo dos campos	Calculado, não editável	`inteiro(slider / 100 × RPM máximo)`. O `900` da captura é ilustrativo; esse texto não é o comando enviado ao motor.
Slider e valor `%` abaixo, inferior esquerdo	Ajuste e leitura do ajuste	Define o `N` da próxima partida do ciclo. Editar não transmite imediatamente; a rotina enviará `cmd=N` e depois `cmd=START`.
Gráfico RPM, superior direito	Leitura	Histórico de `rpm` recebido do motor; mesma telemetria da tela principal.
PROG 1 / PROG 2, botão laranja	Execução local	Inicia ou reinicia o programa correspondente, copiando os tempos e entrando na espera inicial. Exige conexão.
STOP, botão vermelho inferior direito	Cancelamento e comando	Cancela a rotina local e solicita um único `cmd=STOP`, quando conectado.

Para repassar o guia com as imagens, manter estes três arquivos PNG na mesma pasta do Markdown: `screenshot (4).png`, `screenshot (5).png` e `screenshot (6).png`.

Referências: fluxo.h, telas e callbacks e firmware do motor.